INTELIGENCIA DE NEGOCIOS · GRUPO 02

Los ETL de las Dimensiones del Data Mart DM_Cafeteria

Guía de estudio para la exposición: de lo general a lo particular, con cada componente SSIS y cada consulta SQL explicada.

Fuentes analizadas

· `Cafeteria.sql` — base de datos transaccional (origen, OLTP)
· `DM_Cafeteria.sql` — Data Mart en esquema galaxia (destino)
· `BI_Cafeteria_Grupo02.zip` — solución SSIS de Visual Studio 2026 (paquete `Package.dtsx`)

Alcance: los 9 procesos de carga de dimensiones (DimTiempo + 8 flujos de datos). Los hechos se mencionan solo cuando hace falta para entender a las dimensiones.

Contenido

1. **Parte 1 · Lo general**
    1. ¿Qué es un ETL y qué es una dimensión? (explicación sencilla)
    2. El modelo: Cafeteria (origen) → DM_Cafeteria (galaxia)
    3. ¿Por qué se cargan primero las dimensiones?
2. **Parte 2 · El proyecto en Visual Studio 2026**
    1. Archivos de la solución y qué es cada uno
    2. Administradores de conexión y variables
    3. Flujo de control completo del paquete
3. **Parte 3 · El patrón que comparten las 8 dimensiones**
    1. Los 5 componentes del flujo de datos
    2. Propiedades internas de cada componente
    3. SCD Tipo 1, clave sustituta y clave natural
    4. Recorrido de una fila (ejemplo paso a paso)
4. **Parte 4 · Cada dimensión en detalle**
    1. DimTiempo (caso especial: Execute SQL + procedimiento)
    2. DimProducto
    3. DimCliente
    4. DimSucursal
    5. DimEmpleado
    6. DimTipoPedido
    7. DimProveedor
    8. DimVehiculo
    9. DimMetodoPago
5. **Parte 5 · Para la defensa**
    1. Cuadro comparativo de las 9 dimensiones
    2. Lo que *no* se cargó y por qué
    3. Limitaciones honestas y cómo responderlas
    4. Pruebas en vivo para demostrar que funciona
    5. Banco de preguntas probables del docente (con respuesta)
    6. Glosario rápido

> **Cómo leer esta guía.** Cada sección empieza con un recuadro verde **"En simple"** que resume la idea en pocas palabras. Luego viene el detalle técnico. Los recuadros marrones son **preguntas que el docente podría hacer**, y los rojos son **puntos delicados** que conviene tener claros antes de exponer.

Parte 1 · Lo general

1.1 ¿Qué es un ETL y qué es una dimensión?

> **En simple.** La cafetería registra cada venta en su sistema diario (la base **Cafeteria**). Ese sistema está hecho para *operar*, no para *analizar*. El ETL es el "camión de mudanza" que **saca** los datos de ahí (Extract), los **acomoda** en la forma que sirve para analizar (Transform) y los **guarda** en el Data Mart (Load).

ETL significa *Extract – Transform – Load*:

- **Extract (extraer):** leer datos de la base origen `Cafeteria` con una consulta `SELECT`.
- **Transform (transformar):** unir tablas (JOIN), renombrar columnas, calcular campos nuevos (por ejemplo, decidir si un producto es `PLATO` o `BEBIDA`), comparar con lo que ya existe en el destino.
- **Load (cargar):** insertar filas nuevas o actualizar las que cambiaron en `DM_Cafeteria`.

En un Data Mart hay dos tipos de tablas:

Tipo	Qué guarda	Ejemplo en el proyecto
Hechos (Fact)	Los **números** que se miden: cantidades, importes, minutos. Cada fila es un evento del negocio.	`FactVenta`, `FactPago`, `FactDelivery`, `FactRecepcionProducto`
Dimensiones (Dim)	El **contexto** de esos números: quién, qué, dónde, cuándo, cómo. Son los "filtros" y "agrupadores" de los reportes.	`DimProducto`, `DimCliente`, `DimTiempo`, `DimSucursal`…

> **En simple.** Si el gerente pregunta *"¿Cuánto vendimos de cafés en la sucursal California en diciembre?"*, el "cuánto" sale del hecho, y "cafés", "California" y "diciembre" salen de las dimensiones `DimProducto`, `DimSucursal` y `DimTiempo`.

¿Cuál es el propósito de un ETL de dimensión?

Cada ETL de dimensión tiene **un solo fin**: mantener la tabla de dimensión del Data Mart **igual que la realidad del sistema origen**, en un formato cómodo para el análisis. Concretamente, cada vez que se ejecuta:

1. **Agrega** los elementos nuevos (un cliente que se registró hoy, un producto nuevo de la carta).
2. **Corrige** los elementos que cambiaron (el nombre de una sucursal, el rol de un empleado).
3. **No toca** los que siguen iguales (no duplica nada, aunque se ejecute 100 veces).
4. **Asigna una clave sustituta** propia del Data Mart (`ProductoKey`, `ClienteKey`…) que luego usarán los hechos.
5. **Desnormaliza**: junta en una sola fila datos que en el origen estaban en varias tablas (ej. el nombre de la categoría dentro de `DimProducto`).

1.2 El modelo: de Cafeteria a DM_Cafeteria

Cafeteria es una base normalizada (OLTP): muchas tablas pequeñas relacionadas por claves foráneas (Categoria, Marca, TipoPlato, Rol, Cliente, Producto, PedidoVenta, DetallePedidoVenta, Delivery, Pago…).

Tiene triggers y restricciones para que los datos sean consistentes.

DM_Cafeteria es el Data Mart en **esquema galaxia** (también llamado *constelación de hechos*): **4 tablas de hechos** que **comparten** un conjunto de **9 dimensiones** "planas" (sin claves foráneas entre dimensiones). Además tiene la tabla técnica `ETL_Control` y 4 procedimientos almacenados de control.



Figura 1. Arquitectura general: el paquete SSIS lee de Cafeteria y escribe en DM_Cafeteria (misma instancia `localhost`).

Matriz de bus: qué dimensión usa cada hecho

Esta matriz explica **por qué es una galaxia**: varias tablas de hechos reutilizan las mismas dimensiones (*dimensiones conformadas*).

Dimensión	FactVenta	FactRecepcionProducto	FactDelivery	FactPago
DimTiempo	✔	✔	✔ ×3 (pedido, salida, entrega)	✔
DimProducto	✔	✔		
DimCliente	✔		✔	✔
DimSucursal	✔	✔	✔	✔
DimEmpleado	✔ (mozo/vendedor)	✔ (quien recibe)	✔ (repartidor)	
DimTipoPedido	✔			✔
DimProveedor		✔		
DimVehiculo			✔	
DimMetodoPago				✔

Dato útil. En `FactDelivery`, `DimTiempo` aparece 3 veces y `DimEmpleado` aparece como `RepartidorKey`. A esto se le llama **dimensión de rol múltiple** (*role-playing dimension*): una sola tabla física que cumple varios papeles.

1.3 ¿Por qué se cargan primero las dimensiones?

En simple. Un hecho no guarda "Lomo Saltado"; guarda el número `ProductoKey` que lo identifica en `DimProducto`. Si la dimensión todavía no tiene ese producto, el hecho no tiene a quién apuntar. Por eso: **primero dimensiones, después hechos**.

Técnicamente hay dos razones:

1. **Integridad referencial:** en `DM_Cafeteria.sql` cada hecho tiene `FOREIGN KEY` hacia las dimensiones (por ejemplo `FK_FactVenta_Producto REFERENCES dbo.DimProducto(ProductoKey)`). SQL Server no deja insertar un hecho con una clave que no existe en la dimensión.
2. **Búsqueda de claves sustitutas:** los flujos de hechos usan componentes *Búsqueda* (Lookup) contra las dimensiones para traducir `ProductoID` (del origen) a `ProductoKey` (del Data Mart). Esa traducción solo funciona si la dimensión ya está cargada.

Por eso en el paquete el **Contenedor de dimensiones** va antes que el **Contenedor Hechos**, unidos por una restricción de **éxito** (flecha verde).

Parte 2 · El proyecto en Visual Studio 2026

2.1 Archivos de la solución

El zip contiene una solución de **SQL Server Integration Services (SSIS)** creada con **Visual Studio 2026**. Se reconoce porque:

- La solución usa el formato nuevo `.slnx` (XML), propio de las versiones recientes de Visual Studio.
- La carpeta oculta `.vs\BI_Cafeteria_Grupo02.slnx\v18` : "v18" es la versión interna 18 de Visual Studio = **Visual Studio 2026** (VS 2022 es v17).
- El paquete fue guardado por la extensión de SSIS versión `17.0.1016.0` y el proyecto apunta a `TargetServerVersion = SQLServer2025`.

Archivo / carpeta	Para qué sirve
`BI_Cafeteria_Grupo02.slnx`	La **solución**: el "contenedor" que abre Visual Studio. Referencia al proyecto SSIS (el GUID `c9674dcb-…` identifica el tipo de proyecto Integration Services).
`ETL_Cafeteria_Grupo02.dtproj`	El **proyecto** SSIS. Modelo de implementación **Project** (`DeploymentModel = Project`), nivel de protección `EncryptSensitiveWithUserKey` , un único paquete `Package.dtsx` marcado como punto de entrada.
`Package.dtsx`	El **paquete**: aquí está todo el ETL (flujo de control, flujos de datos, consultas SQL). Es un archivo XML de ~1,3 MB. Es el archivo que se abre en el diseñador.
`Project.params`	Parámetros a nivel de proyecto. Está vacío: el proyecto no usa parámetros propios.
`ETL_Cafeteria_Grupo02.database`	Archivo auxiliar que Visual Studio genera para proyectos de BI.
`*.dtproj.user`	Preferencias del usuario (no afectan la ejecución).
`obj\Development\`	Copia intermedia que genera la compilación.
`bin\Development\ETL_Cafeteria_Grupo02.ispac`	El **paquete compilado e implementable**. Es lo que se despliega en el catálogo `SSISDB` de SQL Server si se quisiera programar la ejecución con el Agente SQL.
`UpgradeLog.htm`	Informe de migración. Registra que en algún momento Visual Studio abrió el proyecto sin la extensión de SSIS instalada ("no se encontró la aplicación en la que se basa este tipo de proyecto"). No afecta al paquete.

2.2 Administradores de conexión y variables

Conexiones (Connection Managers)

Nombre	Tipo	Cadena de conexión (resumen)	Rol
`localhost.Cafeteria`	OLE DB	`Data Source=localhost; Initial Catalog=Cafeteria; Provider=MSOLEDBSQL.1; Integrated Security=SSPI`	**Origen**: todas las consultas de extracción.
`localhost.DM_Cafeteria`	OLE DB	`… Initial Catalog=DM_Cafeteria …`	**Destino**: Lookups, INSERT, UPDATE y tareas de control.

- `MSOLEDBSQL` es el proveedor OLE DB moderno de Microsoft para SQL Server.
- `Integrated Security=SSPI` = autenticación de Windows (usa el usuario de Windows que ejecuta el paquete; no hay contraseña guardada).

Variables de usuario

Variable	Valor inicial	Uso
`User::Proceso`	`Carga_DM_Cafeteria`	Nombre del proceso que se registra en `ETL_Control`.
`User::IdControl`	`0`	Recibe el ID de la ejecución que devuelve `SP_ETL_Inicio`. Lo usan los hechos para marcar sus filas.
`User::FechaInicio`	`1/1/1900`	Marca de la última carga correcta (se lee, pero ningún flujo la usa para filtrar).
`User::FechasInsertadas`	`0`	Recibe cuántas fechas insertó **Poblar DimTiempo**.
`User::FilasProcesadas`	`0`	Total de filas de hechos cargadas en la ejecución.
`User::MensajeFinOK` / `MensajeFinError`	Textos fijos	Mensajes que se guardan al cerrar la ejecución.

2.3 Flujo de control del paquete

> **En simple.** El **flujo de control** es la "receta" en orden: abrir el registro de la ejecución → preparar el calendario → cargar las 8 dimensiones → cargar los 4 hechos → contar → cerrar el registro como OK. Si algo falla en cualquier paso, salta a **Fin Error**.

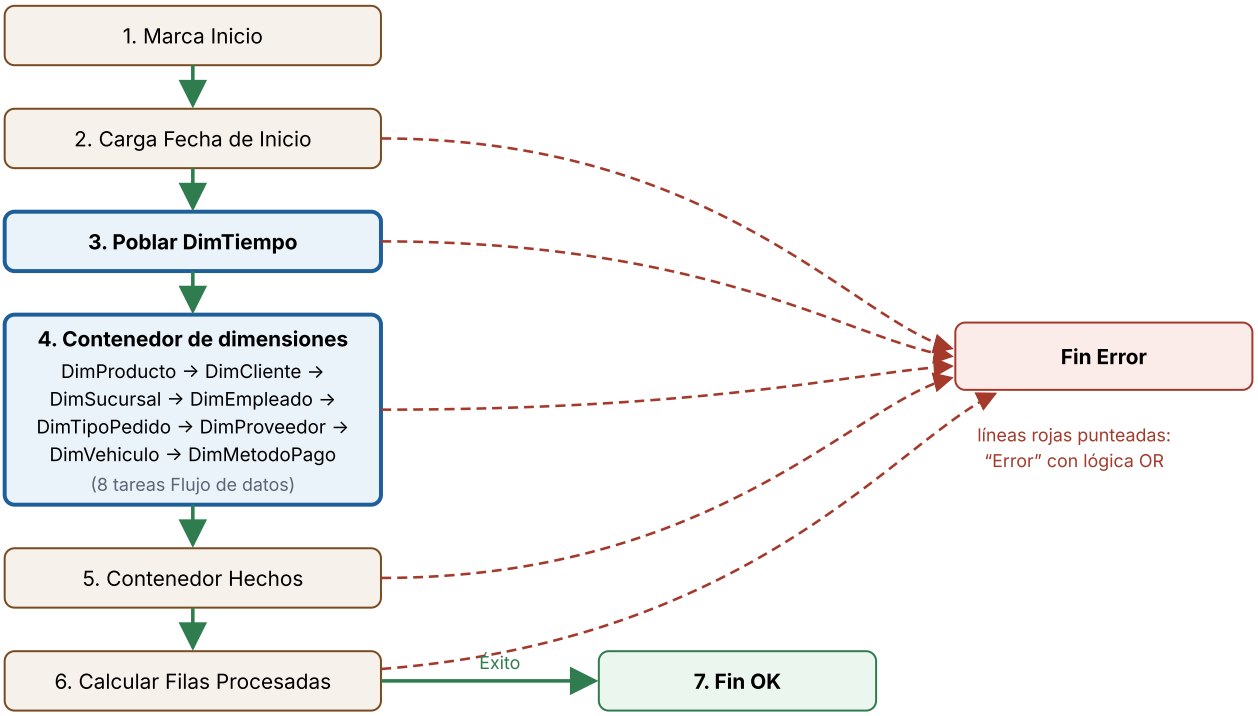


Figura 2. Flujo de control de `Package.dtsx`. En azul, lo que corresponde a las dimensiones.

Detalle de cada tarea del flujo de control

#	Tarea	Tipo	Qué hace
1	Marca Inicio	Ejecutar SQL	`EXEC dbo.SP_ETL_Inicio @Proceso=?, @IdControl=? OUTPUT`. Crea una fila en `ETL_Control` con estado `EN_EJECUCION` y devuelve su ID a `User::IdControl`. Impide dos ejecuciones simultáneas del mismo proceso.
2	Carga Fecha de Inicio	Ejecutar SQL (fila única)	Lee `UltimaFechaCarga` de esa ejecución; si es NULL devuelve 1900-01-01. Resultado → `User::FechaInicio`.
3	**Poblar DimTiempo**	Ejecutar SQL (fila única)	Calcula el rango de fechas del origen y llama a `SP_Poblar_DimTiempo`. **Es el ETL de la dimensión Tiempo** (Parte 4.1).
4	**Contenedor de dimensiones**	Contenedor de secuencias	Agrupa los **8 flujos de datos** de dimensiones, encadenados con restricciones de éxito. Si uno falla, el contenedor falla.
5	Contenedor Hechos	Contenedor de secuencias	FactVenta → FactRecepcionProducto → FactDelivery → FactPago.
6	Calcular Filas Procesadas	Ejecutar SQL	Cuenta las filas de hechos marcadas con el `IdControl` actual.
7	Fin OK	Ejecutar SQL	`EXEC dbo.SP_ETL_Fin_OK`: cierra la ejecución con estado `OK`.
–	Fin Error	Ejecutar SQL	`EXEC dbo.SP_ETL_Fin_Error`: cierra con estado `ERROR`. Le llegan 5 flechas de "Error".

Punto delicado: ¿AND u OR en Fin Error? A **Fin Error** llegan 5 restricciones de fallo. En el XML, esas 5 restricciones **no** tienen `LogicalAnd="True"`, es decir, usan **lógica OR** (en el diseñador se ven como **líneas punteadas**). Eso es correcto: basta con que *una* tarea falle para que se ejecute Fin Error. Si estuvieran en AND, Fin Error solo correría si fallaran las 5 a la vez, lo cual es imposible.

Pregunta probable: "¿Por qué las dimensiones están en un *contenedor de secuencias*?"
Respuesta: Para tratarlas como un bloque: una sola flecha de éxito hacia los hechos y una sola flecha de error hacia Fin Error. Además ordena visualmente el paquete y permite habilitar/deshabilitar o depurar el bloque completo.

Pregunta probable: "¿Por qué las 8 dimensiones están en cadena y no en paralelo, si no dependen entre sí?"
Respuesta: Es cierto que las dimensiones son independientes (no hay FK entre ellas) y podrían ejecutarse en paralelo. Se eligió la cadena por **orden y facilidad de depuración**: si falla una, se sabe exactamente cuál y las siguientes no se ejecutan. Con estos volúmenes (máximo 1000 clientes) el tiempo ganado en paralelo sería mínimo. Además, como cada destino usa `TABLOCK`, la cadena evita competencia de bloqueos.

Parte 3 · El patrón que comparten las 8 dimensiones

> **En simple.** Las 8 dimensiones (todas menos Tiempo) se construyeron con **la misma plantilla de 5 piezas**. Si entiendes una, entiendes todas. La idea es: **"leo el origen, pregunto si ya lo tengo; si no lo tengo lo inserto; si lo tengo, reviso si cambió; si cambió lo actualizo; si no cambió no hago nada".**

3.1 Los 5 componentes del flujo de datos

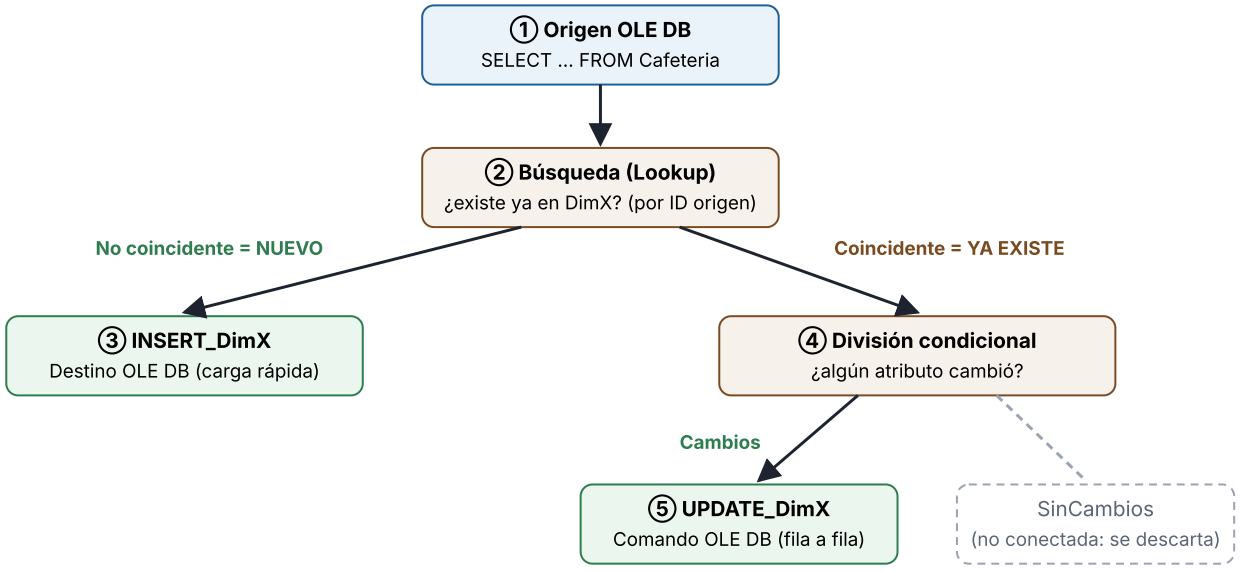


Figura 3. Plantilla común de los 8 flujos de datos de dimensiones (patrón "upsert" = insert + update).

#	Componente SSIS	Función	Conexión
①	Origen de OLE DB	**Extrae**. Ejecuta un `SELECT` en Cafeteria que ya trae las columnas con el nombre y forma que necesita la dimensión (alias, JOIN, CASE).	localhost.Cafeteria
②	Búsqueda (Lookup)	**Compara contra el destino**. Busca cada fila del origen en la dimensión usando la **clave natural** (`…IDOrigen`). Si la encuentra, le pega la clave sustituta y los valores actuales del DM (columnas con sufijo `_DM`).	localhost.DM_Cafeteria
③	Destino de OLE DB (INSERT)	**Carga lo nuevo**. Recibe las filas que el Lookup no encontró y las inserta de una sola vez (carga rápida/bulk).	localhost.DM_Cafeteria
④	División condicional	**Detecta cambios**. Compara columna por columna origen vs. `_DM`. Si alguna es distinta, la fila sale por **Cambios**; si no, por **SinCambios** (salida predeterminada).	– (en memoria)
⑤	Comando de OLE DB (UPDATE)	**Corrige lo modificado**. Ejecuta un `UPDATE … WHERE XKey = ?` por cada fila con cambios.	localhost.DM_Cafeteria

3.2 Propiedades internas de cada componente (lo que hay "dentro")

Esto es lo que se ve al hacer doble clic en cada componente en Visual Studio (o lo que está guardado en el XML del `.dtsx`). Todas las dimensiones usan los **mismos valores**:

① Origen de OLE DB

Propiedad	Valor	Significado
Modo de acceso a datos (`AccessMode`)	`2` = Comando SQL	En lugar de elegir una tabla, se escribe una consulta. Permite hacer JOIN, CASE y alias en la propia extracción.
Administrador de conexión	`localhost.Cafeteria`	Lee de la base transaccional.
Salida	"Salida de origen de OLE DB"	Columnas tipadas: `DT_I4` (int), `DT_WSTR` (nvarchar), `DT_STR` (varchar, página de códigos 1252), `DT_DBDATE` (date), `DT_BOOL` (bit), `DT_I2` (smallint).

② Búsqueda (Lookup)

Propiedad	Valor	Significado
Modo de caché (`CacheType`)	`0` = Caché completa	Antes de que pase la primera fila, SSIS ejecuta la consulta del Lookup y carga **toda la dimensión en memoria**. Luego cada búsqueda es instantánea (no va a la base fila por fila). Ideal para dimensiones pequeñas como estas.
Tipo de conexión	`0` = Administrador OLE DB	La tabla de referencia se lee con `localhost.DM_Cafeteria`.
Filas sin coincidencia (`NoMatchBehavior`)	`1` = Redirigir a la salida no coincidente	**Clave del patrón.** Las filas que no existen en la dimensión no generan error: salen por la "Salida de entradas no coincidentes" hacia el INSERT.
Consulta de referencia (`SqlCommand`)	`SELECT XKey, XIDOrigen, … FROM dbo.DimX;`	Se usa "Usar resultados de una consulta SQL" en vez de elegir la tabla completa, para traer solo las columnas necesarias.
Columna de unión	`XIDOrigen` ↔ `XIDOrigen`	Se une por la **clave natural** (el ID del sistema origen).
Columnas devueltas	`XKey` + atributos con alias `_DM`	El sufijo `_DM` evita choque de nombres: así en el flujo coexisten `Nombre` (origen) y `Nombre_DM` (lo que hay en el Data Mart).
`SqlCommandParam`, `MaxMemoryUsage = 25`	(por defecto)	Solo se usan en caché parcial o sin caché. Con caché completa no aplican.

③ Destino de OLE DB (INSERT_DimX)

Propiedad	Valor	Significado
Modo de acceso (`AccessMode`)	`3` = Tabla o vista – carga rápida	Inserción masiva (bulk insert), mucho más rápida que insertar fila por fila.
Tabla (`OpenRowset`)	`[dbo].[DimX]`	Tabla destino.
Mantener identidad (`FastLoadKeepIdentity`)	`false`	**No** se envía la clave: la columna `XKey` es `IDENTITY(1,1)` y la **genera SQL Server**. Por eso `XKey` no aparece en las columnas mapeadas del INSERT.
Mantener nulos (`FastLoadKeepNulls`)	`false`	Si llega un NULL y la columna tuviera DEFAULT, se usaría el DEFAULT. Las dimensiones no tienen DEFAULT, así que los NULL se guardan como NULL.
Opciones (`FastLoadOptions`)	`TABLOCK, CHECK_CONSTRAINTS`	**TABLOCK**: bloquea la tabla durante la carga (más rápido, menos registro). **CHECK_CONSTRAINTS**: obliga a validar las restricciones `CHECK` del DM durante la carga masiva (sin esta opción la carga rápida las ignora y quedan como "no confiables").
Tamaño máx. de confirmación	`2147483647`	Todo se confirma en un solo lote.
Disposición de error de la entrada	`FailComponent`	Si una fila viola una restricción, el componente **falla** → falla el flujo → falla el contenedor → se ejecuta Fin Error.

④ División condicional

Elemento	Contenido
Salida 1: **Cambios** (orden 0)	Expresión SSIS del tipo `A != A_DM || B != B_DM || …`. El operador `||` es "O": basta un atributo distinto.
Salida predeterminada: **SinCambios**	Recibe todo lo que no cumplió la condición. **No está conectada a nada**, así que esas filas simplemente se descartan (no hay nada que hacer con ellas).

⑤ Comando de OLE DB (UPDATE_DimX)

Propiedad	Significado
`SqlCommand`	`UPDATE dbo.DimX SET col1 = ?, col2 = ? … WHERE XKey = ?;` Cada `?` es un parámetro.
Asignación de columnas	Los `?` se numeran en orden de aparición: `Param_0`, `Param_1`, … En la pestaña "Asignaciones de columnas" se enlaza cada columna del flujo con su parámetro. El último siempre es la clave sustituta (`XKey`, que vino del Lookup).
Comportamiento	Se ejecuta **una vez por cada fila** que llega. Por eso solo se le envían las filas que realmente cambiaron.

3.3 Conceptos que el docente puede preguntar

Clave natural vs. clave sustituta

Concepto	Ejemplo	Quién la crea	Para qué sirve
Clave natural (de negocio)	`ProductoIDOrigen` = `ProductoID` de Cafeteria	El sistema origen	Saber **de qué fila del origen** viene cada fila de la dimensión. Es la que usa el Lookup. Tiene restricción `UNIQUE` en el DM (`UQ_DimProducto_Origen`), así nunca habrá dos filas del mismo producto.
Clave sustituta (surrogate key)	`ProductoKey` (IDENTITY)	El Data Mart	Es la **PK** de la dimensión y la que guardan los hechos. Independiza al DM de los IDs del origen, es un entero compacto (joins rápidos) y permitiría en el futuro historizar (SCD tipo 2).

SCD Tipo 1 (Slowly Changing Dimension)

> **En simple.** Una "dimensión lentamente cambiante" es una dimensión cuyos datos cambian de vez en cuando (un empleado cambia de rol). **Tipo 1 = se sobrescribe**: el dato viejo se reemplaza por el nuevo y no se guarda historia.

El patrón Lookup + División condicional + UPDATE implementa **manualmente un SCD Tipo 1** en las 8 dimensiones. Se nota porque:

- No hay columnas de vigencia (`FechaInicioVigencia`, `FechaFinVigencia`, `EsActual`).
- Cuando algo cambia se hace `UPDATE` sobre la **misma fila** (misma `XKey`), no se inserta una fila nueva.
- La clave natural es `UNIQUE`: solo puede existir una versión por elemento.

Consecuencia: si un empleado pasa de "Mozo" a "Supervisor", todas sus ventas pasadas aparecerán bajo "Supervisor". Para este proyecto es aceptable porque los atributos son descriptivos (nombres, direcciones) y se prioriza que el reporte muestre el dato correcto actual.

¿Por qué no se usó el asistente "Dimensión de variación lenta" de SSIS?

- El asistente SCD de SSIS busca en la dimensión **fila por fila sin caché**, lo que es lento.
- Genera muchos componentes automáticos que son difíciles de modificar después (si se vuelve a abrir el asistente, sobrescribe los cambios).
- El patrón manual con **Lookup en caché completa** es más rápido, más claro de explicar y deja control total sobre qué columnas se comparan.

Idempotencia

Un proceso es **idempotente** cuando ejecutarlo una o varias veces da el mismo resultado. Los ETL de dimensiones lo son: en la 2.ª ejecución el Lookup encuentra todas las filas, la División condicional no detecta cambios y no se inserta ni actualiza nada. Además, la restricción `UNIQUE` sobre `…IDOrigen` es un "cinturón de seguridad" que impediría duplicados aunque el flujo estuviera mal.

3.4 Recorrido de una fila, paso a paso

Tomemos el producto `ProductoID = 16`, *Espresso* (bebida, marca Anyli, caliente, 60 ml).

Momento	Lo que pasa
1.ª ejecución (DM vacío)	① El origen entrega la fila. ② El Lookup busca `ProductoIDOrigen = 16` en `DimProducto`: está vacía → **no coincide**. ③ La fila va a `INSERT_DimProducto`; SQL Server le asigna una `ProductoKey` nueva (IDENTITY).
2.ª ejecución (sin cambios)	② El Lookup encuentra el 16 → **coincide** y agrega `ProductoKey`, `NombreProducto_DM = 'Espresso'`, etc. ④ Todas las comparaciones son iguales → sale por **SinCambios** → se descarta. Nada se escribe.
Alguien cambia el nombre en Cafeteria a "Espresso doble"	② Coincide. ④ `NombreProducto != NombreProducto_DM` es verdadero → sale por **Cambios**. ⑤ `UPDATE dbo.DimProducto SET … NombreProducto = 'Espresso doble' … WHERE ProductoKey = <la misma clave>`. La clave no cambia, así que los hechos que apuntan a ella siguen válidos.
Se agrega un producto 49	② No coincide → ③ se inserta con una clave nueva. Los demás productos no se tocan.

Parte 4 · Cada dimensión en detalle

4.1 DimTiempo — el caso especial

> **En simple.** DimTiempo es el **calendario** del Data Mart: una fila por cada día, con su año, mes, trimestre, nombre del día, si es fin de semana, etc. No existe una tabla "calendario" en Cafeteria, así que **no se copia, se genera**. Por eso no usa un flujo de datos, sino una tarea **Ejecutar SQL** que llama a un procedimiento almacenado.

Propósito

- Tener **todas las fechas** (incluso días sin ventas) para poder analizar por día, semana, mes, trimestre, semestre y año.
- Dar nombres en español (Enero, Lunes...) sin depender del idioma del servidor.
- Debe existir **antes** que todo lo demás: en el flujo de `FactVenta`, la búsqueda `Busqueda_DimTiempo` está configurada para **fallar** si una fecha no existe (`NoMatchBehavior = 0`). Por eso esta tarea va antes incluso del contenedor de dimensiones.

Configuración de la tarea "Poblar DimTiempo"

Propiedad	Valor
Tipo	Tarea Ejecutar SQL (`Microsoft.ExecuteSQLTask`)
Conexión	`localhost.DM_Cafeteria` (pero lee de Cafeteria con nombres de 3 partes: `Cafeteria.dbo.PedidoVenta`)
ResultSet	**Fila única**; la columna 0 (`FechasInsertadas`) se guarda en `User::FechasInsertadas`

Código SQL de la tarea, explicado

```sql
DECLARE @FechaInicio date;
DECLARE @FechaFin date;

;WITH FechasOrigen AS
(
    -- Fecha de los pedidos
    SELECT CONVERT(date, P.FechaHora) AS Fecha
    FROM Cafeteria.dbo.PedidoVenta AS P
    UNION ALL
    -- Fecha de recepción de productos
    SELECT I.Fecha FROM Cafeteria.dbo.IngresoProducto AS I
    UNION ALL
    -- Fecha de salida de los delivery
    SELECT CONVERT(date, D.FechaSalida) FROM Cafeteria.dbo.Delivery AS D
    UNION ALL
    -- Fecha de entrega de los delivery
    SELECT CONVERT(date, D.FechaEntrega) FROM Cafeteria.dbo.Delivery AS D
    WHERE D.FechaEntrega IS NOT NULL
    UNION ALL
    -- Fecha de los pagos
    SELECT CONVERT(date, PG.FechaHora) FROM Cafeteria.dbo.Pago AS PG
)
SELECT
    @FechaInicio = DATEFROMPARTS(YEAR(MIN(Fecha)), 1, 1),
    @FechaFin    = DATEFROMPARTS(YEAR(MAX(Fecha)), 12, 31)
FROM FechasOrigen;

IF @FechaInicio IS NULL OR @FechaFin IS NULL
    THROW 52310, 'No existen fechas en Cafeteria para poblar DimTiempo.', 1;

EXEC dbo.SP_Poblar_DimTiempo @FechaInicio = @FechaInicio, @FechaFin = @FechaFin;
```

Parte	Explicación
`DECLARE @FechaInicio / @FechaFin`	Dos variables para guardar el rango del calendario.
`WITH FechasOrigen AS (…)`	Una **CTE** (expresión de tabla común): una "tabla temporal con nombre" que solo vive en esta consulta.
Los 5 `SELECT`	Reúnen **todas las fechas que usarán los 4 hechos**: pedidos (FactVenta), ingresos (FactRecepcionProducto), salida y entrega (FactDelivery) y pagos (FactPago). Así se garantiza que ningún hecho tenga una fecha que no exista en el calendario.
`CONVERT(date, …)`	Quita la hora: `2026-06-16 20:15:00` → `2026-06-16`. DimTiempo tiene granularidad de **día**.
`UNION ALL`	Apila los resultados. Se usa `ALL` (no quita duplicados) porque es más rápido y solo interesan el MIN y el MAX.
`WHERE FechaEntrega IS NOT NULL`	Un delivery pendiente aún no tiene fecha de entrega.
`DATEFROMPARTS(YEAR(MIN(Fecha)),1,1)`	Toma la fecha más antigua y la lleva al **1 de enero** de ese año.
`DATEFROMPARTS(YEAR(MAX(Fecha)),12,31)`	Toma la más reciente y la lleva al **31 de diciembre**. Resultado: años completos.
`IF … THROW 52310`	Si Cafeteria no tuviera datos, lanza un error controlado → la tarea falla → Fin Error.
`EXEC dbo.SP_Poblar_DimTiempo`	Llama al procedimiento del DM que inserta las fechas.

> **Con los datos del proyecto:** la simulación de Cafeteria va del 01/01/2024 al 30/09/2026, así que el rango queda **01/01/2024 – 31/12/2026** = 366 + 365 + 365 = **1096 fechas**.

Dentro del procedimiento `SP_Poblar_DimTiempo` (definido en DM_Cafeteria.sql)

```
CREATE PROCEDURE dbo.SP_Poblar_DimTiempo @FechaInicio date, @FechaFin date
AS BEGIN
  SET NOCOUNT ON; SET XACT_ABORT ON;
  IF @@TRANCOUNT<>0 OR (2 & @@OPTIONS)=2
     THROW 52200, 'Ejecute el calendario sin transaccion externa y en autocommit.', 1;
  IF @FechaInicio IS NULL OR @FechaFin IS NULL OR @FechaFin<@FechaInicio
     THROW 52201, 'Indique un rango de fechas valido e inclusivo.', 1;
  DECLARE @Dias int = DATEDIFF(DAY,@FechaInicio,@FechaFin)+1;
  IF @Dias>100000 THROW 52202, 'Use rangos de hasta 100000 dias por llamada.', 1;
  BEGIN TRY
    BEGIN TRANSACTION;
    DECLARE @Bloqueo int;
    EXEC @Bloqueo = sys.sp_getapplock @Resource=N'DM_Cafeteria.DimTiempo',
         @LockMode='Exclusive', @LockOwner='Transaction', @LockTimeout=10000;
    IF @Bloqueo<0 THROW 52203, 'No se pudo obtener el bloqueo del calendario.', 1;
    ;WITH dig(n) AS (SELECT n FROM (VALUES(0),(1),(2),(3),(4),(5),(6),(7),(8),(9)) v(n)),
     numeros AS (
       SELECT TOP(@Dias) CONVERT(int, ROW_NUMBER() OVER(ORDER BY (SELECT NULL))-1) AS n
       FROM dig a CROSS JOIN dig b CROSS JOIN dig c CROSS JOIN dig d CROSS JOIN dig e)
    INSERT dbo.DimTiempo(Fecha)
    SELECT DATEADD(DAY,n,@FechaInicio) FROM numeros
    WHERE NOT EXISTS(SELECT 1 FROM dbo.DimTiempo WITH(UPDLOCK,HOLDLOCK)
                     WHERE Fecha = DATEADD(DAY,n,@FechaInicio));
    DECLARE @Insertadas int = @@ROWCOUNT;
    COMMIT TRANSACTION;
    SELECT @Insertadas AS FechasInsertadas;
  END TRY
  BEGIN CATCH
    IF XACT_STATE()<>0 ROLLBACK TRANSACTION; THROW;
  END CATCH;
END;
```

Bloque	Explicación
`SET NOCOUNT ON`	No envía mensajes “(n filas afectadas)”. Importante en SSIS para que el único resultado sea el `SELECT` final.
`SET XACT_ABORT ON`	Si ocurre cualquier error, se aborta y revierte toda la transacción.
`@@TRANCOUNT<>0 OR (2 & @@OPTIONS)=2`	Exige que no haya una transacción abierta desde fuera ni el modo `IMPLICIT_TRANSACTIONS` (bit 2 de `@@OPTIONS`). El SP quiere controlar su propia transacción.
Validación del rango	Fechas no nulas y fin ≥ inicio; máximo 100 000 días por llamada (≈ 273 años).
`sp_getapplock … 'Exclusive'`	Un “candado” con nombre: si dos personas ejecutan el calendario a la vez, una espera (hasta 10 s). Evita inserciones duplicadas simultáneas.
CTE `dig`	Tabla de 10 dígitos (0–9).
CTE `numeros`	Cruza `dig` 5 veces: 10×10×10×10×10 = **100 000** combinaciones. `ROW_NUMBER()-1` las numera 0, 1, 2…; `TOP(@Dias)` se queda solo con las necesarias (1096). Es una “tabla de números” (*tally table*) sin bucles `WHILE`.
`DATEADD(DAY, n, @FechaInicio)`	Convierte cada número en una fecha: 0 → 01/01/2024, 1 → 02/01/2024 …
`WHERE NOT EXISTS (…) WITH(UPDLOCK,HOLDLOCK)`	Solo inserta fechas que **no existen** → el proceso es **idempotente** (en la 2.ª ejecución inserta 0). Las sugerencias de bloqueo evitan que otra sesión inserte la misma fecha mientras se verifica.
`INSERT dbo.DimTiempo(Fecha)`	¡Solo se inserta la columna `Fecha`! Todas las demás son **columnas calculadas PERSISTED** que SQL Server llena solo.
`SELECT @Insertadas AS FechasInsertadas`	El resultado que la tarea SSIS guarda en `User::FechasInsertadas`.

Columnas calculadas de DimTiempo (se llenan solas)

Columna	Fórmula	Ejemplo (Fecha = 2026-10-22)
Anio	`YEAR(Fecha)`	2026
Semestre	`(MONTH(Fecha)+5)/6` (división entera)	(10+5)/6 = 2
Trimestre	`DATEPART(QUARTER,Fecha)`	4
Mes / NombreMes / MesCorto	`MONTH(Fecha)` + `CASE` en español	10 / Octubre / Oct
AnioMes	`YEAR*100+MONTH`	202610 (sirve para ordenar)
AnioTrimestre	texto `'AAAA-T'+trimestre`	2026-T4
AnioMesNombre	`'AAAA - '+NombreMes`	2026 - Octubre
NumeroSemana	`DATEPART(ISO_WEEK,Fecha)`	43 (semana ISO 8601, empieza lunes)
AnioSemanaISO	Corrige el año: si es enero y semana ≥ 52 → año−1; si es diciembre y semana 1 → año+1	2026
DiaMes	`DAY(Fecha)`	22
DiaSemana	`DATEDIFF(DAY,'0001-01-01',Fecha)%7+1`	4 (1=Lunes … 7=Domingo)
NombreDia	mismo cálculo con `CASE`	Jueves
EsFinSemana	`%7 >= 5` → 1	0

El truco del 0001-01-01. El 1 de enero del año 1 fue **lunes**. Contando los días desde esa fecha y tomando el residuo entre 7, se obtiene el día de la semana **sin depender** de `SET DATEFIRST` ni del idioma del servidor (`DATENAME` podría devolver "Monday" en un servidor en inglés).

Pregunta probable: "¿Por qué DimTiempo no usa un flujo de datos como las demás?"
Respuesta: Porque no hay una tabla origen que copiar: el calendario se **genera**. Generar fechas es una operación de conjunto que SQL Server hace en una sola instrucción; un flujo de datos solo añadiría complejidad. El SP además es idempotente y está protegido contra ejecuciones simultáneas.

Pregunta probable: "¿Por qué se generan años completos y no solo los días con ventas?"
Respuesta: Un calendario debe ser **continuo**: los días sin ventas también son información (permite ver "0 ventas") y los reportes por mes/trimestre necesitan todos los días. Cubrir hasta el 31/12 también deja espacio para cargas futuras dentro del año.

4.2 DimProducto — ETL_DimProducto

> **En simple.** Construye el **catálogo de productos** listo para analizar: cada producto con el nombre de su categoría y, según sea **plato** o **bebida**, su tipo de plato o su marca, temperatura y volumen. Es la dimensión **más compleja** porque junta 5 tablas del origen y maneja columnas que pueden ser NULL.

Origen → destino

Tablas origen: `Producto` + `Categoria` + `TipoPlato` + `Marca` + `TemperaturaServicio`. Destino: `DimProducto` (48 filas en la primera carga: 26 platos y 22 bebidas).

Columna DimProducto	Viene de	Nota
ProductoKey	IDENTITY	Clave sustituta (no se mapea)
ProductoIDOrigen	`Producto.ProductoID`	Clave natural (UNIQUE)
SKU	`Producto.SKU`	UNIQUE también en el DM
NombreProducto	`Producto.Nombre`	Renombrado para que sea claro
CategoriaIDOrigen / Categoria	`Producto.CategoriaID` / `Categoria.Nombre`	Desnormalizado
TipoProducto	**calculado** con CASE	'PLATO' o 'BEBIDA'
TipoPlatoIDOrigen / TipoPlato	`Producto.TipoPlatoID` / `TipoPlato.Nombre`	Solo platos (NULL en bebidas)
MarcaIDOrigen / Marca	`Producto.MarcaID` / `Marca.Nombre`	Solo bebidas
TemperaturaIDOrigen / Temperatura	`Producto.TemperaturaID` / `TemperaturaServicio.Nombre`	Solo bebidas
VolumenML	`Producto.VolumenML`	Solo bebidas
Activo	`Producto.Activo`	bit

① Origen de OLE DB — consulta de extracción

```
SELECT
    P.ProductoID   AS ProductoIDOrigen,
    P.SKU,
    P.Nombre       AS NombreProducto,
    P.CategoriaID  AS CategoriaIDOrigen,
    C.Nombre       AS Categoria,
    CASE
        WHEN P.TipoPlatoID IS NOT NULL THEN 'PLATO'
        ELSE 'BEBIDA'
    END AS TipoProducto,
    P.TipoPlatoID   AS TipoPlatoIDOrigen,
    TP.Nombre       AS TipoPlato,
    P.MarcaID       AS MarcaIDOrigen,
    M.Nombre        AS Marca,
    P.TemperaturaID AS TemperaturaIDOrigen,
    TS.Nombre       AS Temperatura,
    P.VolumenML,
    P.Activo
FROM dbo.Producto P
INNER JOIN dbo.Categoria C           ON P.CategoriaID   = C.CategoriaID
LEFT JOIN  dbo.TipoPlato TP          ON P.TipoPlatoID   = TP.TipoPlatoID
LEFT JOIN  dbo.Marca M               ON P.MarcaID       = M.MarcaID
LEFT JOIN  dbo.TemperaturaServicio TS ON P.TemperaturaID = TS.TemperaturaID;
```

Parte	Por qué está así
`AS ProductoIDOrigen`, `AS NombreProducto`…	Los alias hacen que las columnas salgan **con el mismo nombre que en DimProducto**. Así el mapeo del INSERT es directo (SSIS las empareja automáticamente por nombre).
`INNER JOIN Categoria`	Todo producto tiene categoría (`CategoriaID NOT NULL` con FK), así que INNER JOIN no pierde filas.
`LEFT JOIN TipoPlato / Marca / TemperaturaServicio`	Estas FK son **opcionales**: un plato no tiene marca y una bebida no tiene tipo de plato. Con INNER JOIN se perderían productos; con LEFT JOIN se conservan todos y el nombre queda NULL cuando no aplica.
`CASE WHEN P.TipoPlatoID IS NOT NULL THEN 'PLATO' ELSE 'BEBIDA'`	En el origen **no existe** una columna "tipo de producto"; la variante se deduce. La restricción `CK_Producto_Variante` del origen garantiza que si `TipoPlatoID` tiene valor es plato y si no, es bebida (con marca, temperatura y volumen).
Se guardan ID **y** nombre (ej. `CategoriaIDOrigen` y `Categoria`)	El nombre es para los reportes; el ID permite rastrear al origen y detectar cambios aunque dos categorías tuvieran nombres parecidos.
No se extrae `PrecioActual`	El precio cambia con el tiempo y pertenece a cada venta: en el DM vive en el hecho (`FactVenta.PrecioUnitario`). Ponerlo en la dimensión (SCD 1) haría que todas las ventas antiguas "muestren" el precio de hoy.

② Búsqueda (componente "Búsqueda")

```
SELECT ProductoKey, ProductoIDOrigen, SKU, NombreProducto, CategoriaIDOrigen, Categoria,
       TipoProducto, TipoPlatoIDOrigen, TipoPlato, MarcaIDOrigen, Marca,
       TemperaturaIDOrigen, Temperatura, VolumenML, Activo
FROM dbo.DimProducto;
```

Une por `ProductoIDOrigen`. Devuelve `ProductoKey` y todos los atributos con sufijo `_DM`: `SKU_DM`, `NombreProducto_DM`, `CategoriaIDOrigen_DM`, `Categoria_DM`, `TipoProducto_DM`, `TipoPlatoIDOrigen_DM`,

`TipoPlato_DM, MarcaIDOrigen_DM, Marca_DM, TemperaturaIDOrigen_DM, Temperatura_DM, VolumenML_DM, Activo_DM`.

④ División condicional — expresión "Cambios"

```
SKU != SKU_DM
|| NombreProducto != NombreProducto_DM
|| CategoriaIDOrigen != CategoriaIDOrigen_DM
|| Categoria != Categoria_DM
|| TipoProducto != TipoProducto_DM
|| (ISNULL(TipoPlatoIDOrigen) != ISNULL(TipoPlatoIDOrigen_DM))
|| (!ISNULL(TipoPlatoIDOrigen) && !ISNULL(TipoPlatoIDOrigen_DM) && TipoPlatoIDOrigen !=
TipoPlatoIDOrigen_DM)
|| (ISNULL(TipoPlato) != ISNULL(TipoPlato_DM))
|| (!ISNULL(TipoPlato) && !ISNULL(TipoPlato_DM) && TipoPlato != TipoPlato_DM)
|| … (mismo par de condiciones para MarcaIDOrigen, Marca, TemperaturaIDOrigen,
       Temperatura y VolumenML) …
|| Activo != Activo_DM
```

¿Por qué esta expresión es tan larga? (pregunta casi segura)
En las expresiones de SSIS, comparar algo con **NULL da NULL** (no da verdadero ni falso). Y si la condición de una División condicional da NULL, SSIS lanza el error `DTS_E_TXSPLITSTATIC_EXPRESSIONEVALUATEDTONULL` y **el paquete falla**. Como en DimProducto 7 columnas pueden ser NULL (las del plato o las de la bebida), para cada una se usan dos condiciones:

1. `ISNULL(a) != ISNULL(b)` : detecta que uno es NULL y el otro no (por ejemplo, un producto que pasó de plato a bebida).
2. `!ISNULL(a) && !ISNULL(b) && a != b` : solo compara los valores cuando **ambos** tienen dato. Si alguno es NULL, el `&&` se vuelve falso antes de llegar a la comparación.

Si los dos son NULL (ej. la marca de un plato), ambas condiciones son falsas → no es un cambio. En las otras dimensiones no hace falta porque todas sus columnas son `NOT NULL`.

③ INSERT_DimProducto

Mapea 14 columnas: `ProductoIDOrigen, SKU, NombreProducto, CategoriaIDOrigen, Categoria, TipoProducto, TipoPlatoIDOrigen, TipoPlato, MarcaIDOrigen, Marca, TemperaturaIDOrigen, Temperatura, VolumenML, Activo`. Con `CHECK_CONSTRAINTS` se valida `CK_DimProducto_Variante` : un PLATO debe tener tipo de plato y NULL en marca/temperatura/volumen; una BEBIDA lo contrario y volumen > 0.

⑤ UPDATE_DimProducto

```
UPDATE dbo.DimProducto
SET SKU = ?, NombreProducto = ?, CategoriaIDOrigen = ?, Categoria = ?, TipoProducto = ?,
    TipoPlatoIDOrigen = ?, TipoPlato = ?, MarcaIDOrigen = ?, Marca = ?,
    TemperaturaIDOrigen = ?, Temperatura = ?, VolumenML = ?, Activo = ?
WHERE ProductoKey = ?;
```

Parámetro	Columna del flujo	Parámetro	Columna del flujo
Param_0	SKU	Param_7	MarcaIDOrigen
Param_1	NombreProducto	Param_8	Marca
Param_2	CategoriaIDOrigen	Param_9	TemperaturaIDOrigen
Param_3	Categoria	Param_10	Temperatura
Param_4	TipoProducto	Param_11	VolumenML
Param_5	TipoPlatoIDOrigen	Param_12	Activo
Param_6	TipoPlato	Param_13	**ProductoKey** (del Lookup)

Se actualizan los valores **del origen** (no los `_DM`) y se localiza la fila por la **clave sustituta**, que es la PK (búsqueda más rápida que por `ProductoIDOrigen`). `ProductoIDOrigen` no se actualiza: es la identidad del producto.

> **Pregunta probable:** "¿Por qué DimProducto no tiene una DimCategoria aparte?"
> **Respuesta:** Porque el DM es una **galaxia de dimensiones planas** (estrella, no copo de nieve). Meter la categoría, marca y tipo de plato dentro de DimProducto evita joins extra en los reportes y es lo que recomienda Kimball. La redundancia (el nombre de la categoría se repite) no importa en un DM de solo lectura.

4.3 DimCliente — ETL_DimCliente

> **En simple.** Lista a todos los clientes con **un solo nombre** para mostrar y su **tipo** (persona natural o empresa). En el origen, una persona tiene nombre y apellido con DNI, y una empresa tiene razón social con RUC; la dimensión unifica ambos casos en una sola columna.

Origen: `Cliente` (1000 filas: 897 naturales y 103 empresas). Destino: `DimCliente`.

Columna DimCliente	Viene de
ClienteKey	IDENTITY
ClienteIDOrigen	`Cliente.ClienteID`
NombreCliente	**calculado**: `Nombre + ' ' + Apellido` o `RazonSocial`
TipoCliente	**calculado**: 'NATURAL' o 'EMPRESA'
FechaRegistro	`Cliente.FechaRegistro`

① Origen de OLE DB

```
SELECT
    C.ClienteID AS ClienteIDOrigen,
    CASE
        WHEN C.DNI IS NOT NULL
            THEN CONCAT(C.Nombre, N' ', C.Apellido)
        ELSE C.RazonSocial
    END AS NombreCliente,
    CASE
        WHEN C.DNI IS NOT NULL THEN 'NATURAL'
        ELSE 'EMPRESA'
    END AS TipoCliente,
    C.FechaRegistro
FROM dbo.Cliente C;
```

Parte	Explicación
`CASE WHEN C.DNI IS NOT NULL`	El DNI decide el tipo. La restricción `CK_Cliente_Variante` del origen asegura que un cliente tiene **o** DNI+Nombre+Apellido, **o** RUC+RazonSocial, nunca ambos ni ninguno.
`CONCAT(C.Nombre, N' ', C.Apellido)`	Une nombre y apellido con un espacio. Se usa `CONCAT` en vez de `+` porque `CONCAT` trata NULL como texto vacío (más seguro). La `N` indica texto Unicode.
Tamaño `nvarchar(181)` en el DM	80 (Nombre) + 1 (espacio) + 100 (Apellido) = 181. Es el máximo posible; RazonSocial (150) también entra.
Columnas que **no** se cargan	`Celular`, `DNI`, `RUC`: son datos personales/identificadores que no se usan para agrupar ventas. Se evita copiar información sensible al DM (minimización de datos).

② Búsqueda

```
SELECT ClienteKey, ClienteIDOrigen, NombreCliente, TipoCliente, FechaRegistro
FROM dbo.DimCliente;
```

Une por `ClienteIDOrigen`; devuelve `ClienteKey, NombreCliente_DM, TipoCliente_DM, FechaRegistro_DM`. Con 1000 clientes, la caché completa ocupa muy poca memoria.

④ División condicional

```
NombreCliente != NombreCliente_DM || TipoCliente != TipoCliente_DM || FechaRegistro != FechaRegistro_DM
```

Sin `ISNULL` porque las tres columnas son siempre no nulas (el CASE siempre devuelve un valor).

③ INSERT_DimCliente y ⑤ UPDATE_DimCliente

INSERT: `ClienteIDOrigen, NombreCliente, TipoCliente, FechaRegistro`. Se valida `CK_DimCliente_Tipo` (solo 'NATURAL'/'EMPRESA') y `CK_DimCliente_Nombre` (no vacío).

```
UPDATE dbo.DimCliente
SET NombreCliente = ?, TipoCliente = ?, FechaRegistro = ?   -- Param_0, Param_1, Param_2
WHERE ClienteKey = ?;                                         -- Param_3
```

Pregunta probable: "¿Qué pasa si una empresa se registra por error como persona y luego se corrige?"
Respuesta: En la siguiente ejecución el Lookup la encuentra por `ClienteIDOrigen`, la División condicional detecta que cambió `TipoCliente` y `NombreCliente`, y el UPDATE corrige la misma fila (SCD 1). Sus ventas pasadas pasan a verse como "EMPRESA".

4.4 DimSucursal — ETL_DimSucursal

> **En simple.** Lista las **sedes** de la cafetería (Trujillo Centro, Trujillo California, Trujillo El Recreo) con su dirección y fecha de apertura. Es la dimensión que usan **los 4 hechos**: es la más "conformada" del modelo.

Origen: `Sucursal` (3 filas). Destino: `DimSucursal`. Es una copia casi directa: no hay JOIN ni CASE, solo renombres.

① Origen (componente "Origen_Cafeteria")

```
SELECT
    S.SucursalID AS SucursalIDOrigen,
    S.Nombre     AS NombreSucursal,
    S.Direccion,
    S.FechaApertura
FROM dbo.Sucursal AS S;
```

② Búsqueda ("Busqueda_DM_Cafeteria")

```
SELECT SucursalKey, SucursalIDOrigen, NombreSucursal, Direccion, FechaApertura
FROM dbo.DimSucursal;
```

Devuelve `SucursalKey, NombreSucursal_DM, Direccion_DM, FechaApertura_DM`.

④ División condicional ("Cambios_DimSucursal")

```
NombreSucursal != NombreSucursal_DM || Direccion != Direccion_DM || FechaApertura !=
FechaApertura_DM
```

③ INSERT_DimSucursal y ⑤ UPDATE_DimSucursal

INSERT: `SucursalIDOrigen, NombreSucursal, Direccion, FechaApertura`.

```
UPDATE dbo.DimSucursal
SET NombreSucursal = ?, Direccion = ?, FechaApertura = ?   -- Param_0..2
WHERE SucursalKey = ?;                                      -- Param_3
```

> **Detalle de nombres.** A partir de esta dimensión el grupo renombró los componentes con nombres descriptivos (`Origen_Cafeteria`, `Busqueda_DM_Cafeteria`, `Cambios_DimSucursal`). En DimProducto y DimCliente quedaron los nombres por defecto ("Origen de OLE DB", "Búsqueda", "División condicional"). El funcionamiento es idéntico.

> **Pregunta probable:** "¿Por qué no se cargan las mesas y cajas en DimSucursal?"
> **Respuesta:** Porque ningún hecho se analiza por mesa o caja (FactVenta tiene granularidad de línea de pedido y no guarda la mesa). Mesa y Caja tienen su propio nivel de detalle; meterlas en DimSucursal cambiaría su granularidad (habría varias filas por sucursal).

4.5 DimEmpleado — ETL_DimEmpleado

> **En simple.** Lista a los **trabajadores** con su rol (Administrador, Mozo, Cajero, Repartidor, Supervisor). La misma dimensión sirve para saber quién vendió, quién recibió mercadería y quién repartió un delivery.

Origen: `Empleado` + `Rol` (21 empleados). Destino: `DimEmpleado`.

① Origen

```
SELECT
    E.EmpleadoID AS EmpleadoIDOrigen,
    E.Nombre,
    E.Apellido,
    E.RolID      AS RolIDOrigen,
    R.Nombre     AS Rol,
    E.FechaIngreso,
    E.Activo
FROM dbo.Empleado AS E
INNER JOIN dbo.Rol AS R ON E.RolID = R.RolID;
```

- `INNER JOIN Rol`: se **desnormaliza** el nombre del rol dentro de la dimensión. Es seguro usar INNER porque `Empleado.RolID` es `NOT NULL` con FK: todo empleado tiene rol.
- Se guardan `Nombre` y `Apellido` por separado (a diferencia de DimCliente) porque aquí no hay dos variantes que unificar.
- `Activo` permite filtrar empleados que ya no trabajan sin borrarlos (sus ventas históricas siguen necesitando la fila).

② Búsqueda

```
SELECT EmpleadoKey, EmpleadoIDOrigen, Nombre, Apellido, RolIDOrigen, Rol, FechaIngreso, Activo
FROM dbo.DimEmpleado;
```

Devuelve `EmpleadoKey, Nombre_DM, Apellido_DM, RolIDOrigen_DM, Rol_DM, FechaIngreso_DM, Activo_DM`.

④ División condicional ("Cambios_DimEmpleado")

```
Nombre != Nombre_DM || Apellido != Apellido_DM || RolIDOrigen != RolIDOrigen_DM
|| Rol != Rol_DM || FechaIngreso != FechaIngreso_DM || Activo != Activo_DM
```

③ INSERT y ⑤ UPDATE

INSERT: `EmpleadoIDOrigen, Nombre, Apellido, RolIDOrigen, Rol, FechaIngreso, Activo`.

```
UPDATE dbo.DimEmpleado
SET Nombre = ?, Apellido = ?, RolIDOrigen = ?, Rol = ?, FechaIngreso = ?, Activo = ?  -- Param_0..5
WHERE EmpleadoKey = ?;                                                                 -- Param_6
```

Pregunta probable: "¿Por qué no hay una DimRepartidor?"
Respuesta: Porque un repartidor **es un empleado** con rol "Repartidor". Se reutiliza DimEmpleado como **dimensión de rol**: en FactDelivery la columna se llama `RepartidorKey` pero referencia a `DimEmpleado(EmpleadoKey)`. Así no se duplican datos.

Pregunta probable: "¿Y si se renombra un rol, por ejemplo 'Mozo' a 'Mesero'?"
Respuesta: La consulta origen trae el nuevo nombre para todos los mozos; la División condicional detecta `Rol != Rol_DM` en cada uno y el UPDATE los corrige. Es una ventaja de guardar también `Rol` (texto) y compararlo.

4.6 DimTipoPedido — ETL_DimTipoPedido

> **En simple.** Guarda los **canales de venta**: En Mesa, Delivery y Para Llevar. Permite comparar ventas y cobros por canal.

Origen: `TipoPedido` (3 filas). Destino: `DimTipoPedido`. Usada por FactVenta y FactPago.

```
-- ① Origen_Cafeteria
SELECT TP.TipoPedidoID AS TipoPedidoIDOrigen,
       TP.Nombre       AS NombreTipoPedido
FROM dbo.TipoPedido AS TP;

-- ② Busqueda_DM_Cafeteria (une por TipoPedidoIDOrigen; devuelve TipoPedidoKey,
NombreTipoPedido_DM)
SELECT TipoPedidoKey, TipoPedidoIDOrigen, NombreTipoPedido FROM dbo.DimTipoPedido;

-- ④ Cambios_DimTipoPedido
NombreTipoPedido != NombreTipoPedido_DM

-- ⑤ "Comando de OLE DB" (es el UPDATE; conservó el nombre por defecto)
UPDATE dbo.DimTipoPedido SET NombreTipoPedido = ?   -- Param_0
WHERE TipoPedidoKey = ?;                             -- Param_1
```

③ `INSERT_DimTipoPedido` mapea `TipoPedidoIDOrigen, NombreTipoPedido`.

> **Pregunta probable:** "¿Para qué una dimensión de solo 3 filas? ¿No basta con guardar el texto en el hecho?"
> **Respuesta:** Una dimensión pequeña sigue siendo útil: el hecho guarda un entero (4 bytes) en vez de un texto repetido miles de veces, el nombre se corrige en un solo lugar y la herramienta de reportes la muestra como un filtro limpio. Además es **conformada**: la comparten FactVenta y FactPago, lo que permite cruzar ventas y cobros por el mismo canal.

4.7 DimProveedor — ETL_DimProveedor

> **En simple.** Lista a los **proveedores** que abastecen a la cafetería, con su RUC y razón social. Solo la usa el hecho de recepción de productos (compras).

Origen: `Proveedor` (8 filas). Destino: `DimProveedor`.

```sql
-- ① Origen_Cafeteria
SELECT P.ProveedorID AS ProveedorIDOrigen,
       P.RUC,
       P.RazonSocial
FROM dbo.Proveedor AS P;

-- ② Busqueda_DM_Cafeteria (devuelve ProveedorKey, RUC_DM, RazonSocial_DM)
SELECT ProveedorKey, ProveedorIDOrigen, RUC, RazonSocial FROM dbo.DimProveedor;

-- ④ Cambios_DimProveedor
RUC != RUC_DM || RazonSocial != RazonSocial_DM

-- ⑤ UPDATE_DimProveedor
UPDATE dbo.DimProveedor SET RUC = ?, RazonSocial = ?   -- Param_0, Param_1
WHERE ProveedorKey = ?;                                -- Param_2
```

- ③ El INSERT se llama **"Destino de OLE DB"** (nombre por defecto) y mapea `ProveedorIDOrigen, RUC, RazonSocial`.
- El RUC es `char(11)` (en SSIS `DT_STR` de longitud 11). En el DM hay `CK_DimProveedor_RUC`: 11 caracteres y todos dígitos (`NOT LIKE '%[^0-9]%'` = "no contiene ningún carácter que no sea dígito"). Gracias a `CHECK_CONSTRAINTS` esa regla se valida también en la carga rápida.
- A diferencia de DimCliente, aquí **sí** se carga el RUC: es un dato público de la empresa y útil para identificarla en reportes de compras.

4.8 DimVehiculo — ETL_DimVehiculo

> **En simple.** Lista los **vehículos** (motos) con que se reparten los delivery, identificados por placa. Solo la usa FactDelivery, para analizar tiempos de entrega por vehículo.

Origen: `Vehiculo` (7 filas, placas SIM-101 a SIM-107). Destino: `DimVehiculo`.

```
-- ① Origen_Cafeteria
SELECT V.VehiculoID AS VehiculoIDOrigen,
       V.Placa,
       V.Descripcion
FROM dbo.Vehiculo AS V;

-- ② Busqueda_DM_Cafeteria (devuelve VehiculoKey, Placa_DM, Descripcion_DM)
SELECT VehiculoKey, VehiculoIDOrigen, Placa, Descripcion FROM dbo.DimVehiculo;

-- ④ Cambios_DimVehiculo  (salidas: "Cambio" y "SinCambio")
Placa != Placa_DM || Descripcion != Descripcion_DM

-- ⑤ UPDATE_DimVehiculo
UPDATE dbo.DimVehiculo SET Placa = ?, Descripcion = ?   -- Param_0, Param_1
WHERE VehiculoKey = ?;                                  -- Param_2
```

③ `INSERT_DimVehiculo` mapea `VehiculoIDOrigen, Placa, Descripcion`. La placa tiene `UNIQUE` en origen y en DM.

> **Detalle curioso.** En DimVehiculo y DimMetodoPago, las salidas internas aparecen en inglés (`Lookup Match Output`, `Lookup No Match Output`, `OLE DB Source Output`) en lugar de español ("Salida de entradas coincidentes de búsqueda"). Esto pasa cuando esos componentes se crean con una instalación/idioma distinto de la extensión SSIS. **No cambia nada** del funcionamiento: son solo etiquetas.

4.9 DimMetodoPago — ETL_DimMetodoPago

> **En simple.** Lista las **formas de pago**: Efectivo, Visa, Mastercard, Yape y Plin. Solo la usa FactPago, para saber cuánto se cobra por cada medio (incluye pagos mixtos: un pedido puede tener 2 pagos con métodos distintos).

Origen: `MetodoPago` (5 filas). Destino: `DimMetodoPago`. Es la última dimensión de la cadena.

```
-- ① Origen_Cafeteria
SELECT MP.MetodoPagoID AS MetodoPagoIDOrigen,
       MP.Nombre       AS NombreMetodoPago
FROM dbo.MetodoPago AS MP;

-- ② Busqueda_DM_Cafeteria (devuelve MetodoPagoKey, NombreMetodoPago_DM)
SELECT MetodoPagoKey, MetodoPagoIDOrigen, NombreMetodoPago FROM dbo.DimMetodoPago;

-- ④ Cambios_DimMetodoPago
NombreMetodoPago != NombreMetodoPago_DM

-- ⑤ UPDATE_DimMetodoPago
UPDATE dbo.DimMetodoPago SET NombreMetodoPago = ?   -- Param_0
WHERE MetodoPagoKey = ?;                             -- Param_1
```

③ `INSERT_DimMetodoPago` mapea `MetodoPagoIDOrigen, NombreMetodoPago`.

Pregunta probable: “Si la cafetería agrega ‘Tarjeta Oh’ como método 6, ¿qué hay que cambiar en el ETL?”
Respuesta: Nada. En la próxima ejecución el Lookup no encontrará el ID 6, la fila saldrá por “No coincidente” y se insertará con una nueva `MetodoPagoKey`. Luego FactPago ya podrá usarla.

Parte 5 · Para la defensa

5.1 Cuadro comparativo de las 9 dimensiones

Dimensión	Tarea SSIS	Tablas origen	Transformación clave	Atributos comparados	Filas (1.ª carga)	Hechos que la usan
DimTiempo	Ejecutar SQL + SP	(fechas de PedidoVenta, IngresoProducto, Delivery, Pago)	Genera calendario; columnas calculadas	– (NOT EXISTS por Fecha)	1096	Los 4
DimProducto	Flujo de datos	Producto, Categoria, TipoPlato, Marca, TemperaturaServicio	1 INNER + 3 LEFT JOIN; CASE PLATO/BEBIDA	13 (con ISNULL)	48	Venta, Recepción
DimCliente	Flujo de datos	Cliente	CASE + CONCAT: nombre y tipo	3	1000	Venta, Delivery, Pago
DimSucursal	Flujo de datos	Sucursal	Renombres	3	3	Los 4
DimEmpleado	Flujo de datos	Empleado, Rol	INNER JOIN Rol	6	21	Venta, Recepción, Delivery
DimTipoPedido	Flujo de datos	TipoPedido	Renombres	1	3	Venta, Pago
DimProveedor	Flujo de datos	Proveedor	Copia directa	2	8	Recepción
DimVehiculo	Flujo de datos	Vehiculo	Copia directa	2	7	Delivery
DimMetodoPago	Flujo de datos	MetodoPago	Renombres	1	5	Pago

Los conteos se dedujeron del script `Cafeteria.sql` (datos fijos + simulación determinista). Conviene confirmarlos con `SELECT COUNT(*)` tras ejecutar el paquete.

5.2 Lo que no se cargó como dimensión, y por qué

Tabla origen	Motivo
`EstadoPedido`	FactVenta y FactPago solo cargan pedidos **Completados** (`WHERE P.EstadoPedidoID = 2`). Una dimensión de estado tendría siempre el mismo valor; el propio script del DM lo comenta.
`Categoria`, `Marca`, `TipoPlato`, `TemperaturaServicio`	No son dimensiones aparte: se **desnormalizaron** dentro de DimProducto (modelo estrella, no copo de nieve).
`Rol`	Desnormalizado dentro de DimEmpleado.
`Mesa`, `Caja`, `CajeroCaja`, `Turno`, `EmpleadoTurno`	Ningún hecho se analiza a ese nivel. El origen no registra en el pago qué cajero cobró, así que no hay cómo relacionarlos.
`ComprobantePago`	Sus datos (serie, número, fecha) quedaron como **dimensión degenerada** dentro de FactPago (columnas en el hecho, sin tabla propia).

5.3 Limitaciones honestas (y cómo responderlas)

Un buen expositor conoce los límites de su diseño. Si el docente los señala, estas son respuestas sólidas:

Observación	Respuesta
El UPDATE con "Comando de OLE DB" es fila a fila	Correcto: ejecuta un UPDATE por cada fila cambiada. Con dimensiones pequeñas y pocos cambios es aceptable. Para volúmenes grandes la alternativa es enviar los cambios a una tabla de staging y hacer un único `UPDATE … FROM` o `MERGE` con una tarea Ejecutar SQL.
No se guarda historia (SCD 1)	Fue una decisión: se prioriza mostrar el dato actual. Si se necesitara historia (por ejemplo del rol del empleado), se aplicaría SCD Tipo 2: agregar `FechaInicio`, `FechaFin`, `EsActual`, quitar el `UNIQUE` de la clave natural e insertar una nueva versión en vez de actualizar.
No hay "miembro desconocido" (fila −1 / "No especificado")	No hace falta porque el origen garantiza con FK y CHECK que todo hecho tiene producto, cliente, sucursal, etc. Si en el futuro llegaran datos incompletos, se agregaría una fila especial en cada dimensión para no perder hechos.
Si se borra algo en el origen, la dimensión no lo borra	Es lo deseable: los hechos históricos siguen necesitando esa fila. Para "bajas" se usa el atributo `Activo` (Producto y Empleado).
Las dimensiones no se registran en ETL_Control	Las dimensiones no tienen columna `IdControlCarga`; `FilasProcesadas` cuenta solo hechos. Se podría mejorar contando filas insertadas/actualizadas con la transformación **Recuento de filas** (Row Count) en cada flujo.
La comparación de textos distingue mayúsculas	En SSIS, `"café" != "Café"` es verdadero. Es intencional: si alguien corrige una tilde o mayúscula, se quiere ver la corrección en el DM.
Ambas bases deben estar en la misma instancia	"Poblar DimTiempo" usa nombres de 3 partes (`Cafeteria.dbo.…`) desde la conexión al DM. Si las bases estuvieran en servidores distintos habría que usar un servidor vinculado o leer las fechas con un flujo de datos.

5.4 Pruebas en vivo para demostrar que funciona

Si el docente pide demostrar el comportamiento, estas pruebas son rápidas y muy convincentes:

Prueba 1 · Idempotencia (no duplica)

```
-- Ejecutar el paquete 2 veces y luego:
SELECT 'DimProducto', COUNT(*) FROM DM_Cafeteria.dbo.DimProducto UNION ALL
SELECT 'DimCliente',  COUNT(*) FROM DM_Cafeteria.dbo.DimCliente  UNION ALL
SELECT 'DimTiempo',   COUNT(*) FROM DM_Cafeteria.dbo.DimTiempo;
-- Los conteos no cambian (48, 1000, 1096). En la 2.ª ejecución, en la pestaña
-- "Progreso"/"Flujo de datos" se ve 0 filas hacia INSERT y 0 hacia UPDATE.
```

Prueba 2 · Actualización (SCD 1)

```
UPDATE Cafeteria.dbo.Sucursal SET Nombre = N'Trujillo Centro Histórico' WHERE SucursalID = 1;
-- Ejecutar solo ETL_DimSucursal (clic derecho > Ejecutar tarea). Se ve 1 fila por "Cambios".
SELECT SucursalKey, SucursalIDOrigen, NombreSucursal FROM DM_Cafeteria.dbo.DimSucursal;
-- Misma SucursalKey, nombre nuevo.
```

Prueba 3 · Inserción de un nuevo miembro

```
INSERT Cafeteria.dbo.MetodoPago(MetodoPagoID, Nombre) VALUES (6, N'Tarjeta Oh');
-- Ejecutar ETL_DimMetodoPago: 1 fila por "No coincidente" hacia INSERT_DimMetodoPago.
SELECT * FROM DM_Cafeteria.dbo.DimMetodoPago;
```

> **Antes de la exposición:** después de las pruebas, revierte los cambios en Cafeteria (vuelve a poner el nombre original, borra el método 6) para no dejar datos de prueba. Recuerda que las tablas de pedidos tienen triggers que impiden modificar pedidos cerrados; para las pruebas usa tablas de catálogo (Sucursal, MetodoPago, Producto...), que no tienen triggers.

5.5 Banco de preguntas probables del docente

1. ¿Qué es un ETL y cuál es el objetivo de los ETL de dimensiones?

Extraer, transformar y cargar. Los de dimensiones mantienen las tablas de contexto del DM sincronizadas con el origen: insertan lo nuevo, actualizan lo que cambió, asignan claves sustitutas y desnormalizan catálogos.

2. ¿Qué herramienta usaron?

SQL Server Integration Services (SSIS) en Visual Studio 2026 (solución .slnx, carpeta .vs\v18), extensión SSIS 17.0, servidor de destino SQL Server 2025, modelo de implementación por proyecto.

3. ¿Qué diferencia hay entre el flujo de control y el flujo de datos?

El **flujo de control** ordena tareas (Ejecutar SQL, contenedores, flujos de datos) y define qué pasa si tienen éxito o fallan. El **flujo de datos** está dentro de una tarea "Flujo de datos" y mueve filas por componentes (origen → transformaciones → destino) en memoria.

4. ¿Por qué primero DimTiempo y luego el contenedor de dimensiones?

Porque todos los hechos buscan su fecha en DimTiempo; en FactVenta esa búsqueda está configurada para fallar si la fecha no existe. Y las dimensiones van antes que los hechos por las FK y porque los hechos necesitan sus claves sustitutas.

5. ¿Qué hace exactamente el componente Búsqueda (Lookup)?

Carga la dimensión en memoria (caché completa) y, por cada fila del origen, busca su clave natural. Si existe, le agrega la clave sustituta y los valores actuales (`_DM`) y la envía a la salida coincidente; si no existe, la redirige a la salida no coincidente (filas nuevas).

6. ¿Por qué caché completa?

Las dimensiones son pequeñas (máximo 1000 filas); cargarlas una vez en memoria hace la búsqueda instantánea y evita miles de consultas a la base.

7. ¿Qué pasa con las filas que salen por "SinCambios"?

Nada: la salida no está conectada y se descartan, porque la dimensión ya tiene esos datos iguales.

8. ¿Por qué el INSERT no mapea la columna Key?

Porque es `IDENTITY(1,1)` y `FastLoadKeepIdentity=false`: la genera SQL Server.

9. ¿Qué significan TABLOCK y CHECK_CONSTRAINTS?

TABLOCK bloquea la tabla para una carga masiva más rápida. CHECK_CONSTRAINTS hace que la carga rápida valide las restricciones CHECK del DM (por defecto el bulk insert las salta).

10. ¿Qué significan los signos "?" en el UPDATE?

Son parámetros posicionales del Comando OLE DB (`Param_0`, `Param_1`...), que se enlazan con columnas del flujo en la pestaña de asignaciones. El último es siempre la clave sustituta del WHERE.

11. ¿Por qué el UPDATE usa XKey y no XIDOrigen en el WHERE?

Porque XKey es la clave primaria (índice agrupado): la búsqueda es la más eficiente, y ya la tenemos gracias al Lookup.

12. ¿Qué tipo de SCD implementaron?

Tipo 1 (sobrescritura), de forma manual con Lookup + División condicional + Comando OLE DB, sin usar el asistente SCD.

13. ¿Por qué la expresión de DimProducto usa ISNULL?

Porque en SSIS comparar con NULL devuelve NULL y una condición NULL en la División condicional hace fallar el componente. DimProducto es la única dimensión con columnas opcionales (plato/bebida).

14. ¿Por qué LEFT JOIN en DimProducto e INNER JOIN en DimEmpleado?

En Producto, TipoPlato, Marca y Temperatura son FK opcionales (NULL según la variante): INNER JOIN eliminaría productos. En Empleado, RolID es obligatorio: INNER JOIN es seguro.

15. ¿Cómo se sabe si un producto es plato o bebida?

Con `CASE WHEN TipoPlatoID IS NOT NULL THEN 'PLATO' ELSE 'BEBIDA'`, apoyado en la restricción de variante del origen.

16. ¿Cómo se arma el nombre del cliente?

Si tiene DNI: `CONCAT(Nombre,' ',Apellido)` y tipo NATURAL; si no: `RazonSocial` y tipo EMPRESA.

17. ¿Cuántas filas tendrá DimTiempo y por qué?

1096: del 01/01/2024 al 31/12/2026, porque se toma el año de la fecha mínima y máxima del origen y se completan los años.

18. ¿Qué pasa si se ejecuta el paquete dos veces?

Nada se duplica: el Lookup encuentra todo, no hay cambios, y el SP del calendario inserta 0 fechas (NOT EXISTS). Además las claves naturales tienen UNIQUE.

19. ¿Qué pasa si falla una dimensión?

Las entradas de destino y comando están en `FailComponent`: el flujo falla, se detiene la cadena, el contenedor falla y por la restricción de error se ejecuta Fin Error, que marca la ejecución como ERROR en ETL_Control. Los hechos no se cargan.

20. ¿Las flechas hacia Fin Error son AND u OR?

OR (líneas punteadas, sin LogicalAnd): basta con que una tarea falle.

21. ¿Qué es una dimensión conformada?

Una dimensión compartida por varios hechos con el mismo significado y claves. Ej.: DimSucursal y DimTiempo se usan en los 4 hechos; por eso el modelo es una galaxia.

22. ¿Qué es una dimensión de rol?

Una dimensión física usada con distintos papeles: DimTiempo como fecha de pedido, salida y entrega en FactDelivery; DimEmpleado como repartidor.

23. ¿Por qué guardar el ID origen en la dimensión?

Es la clave natural: permite al Lookup saber si el elemento ya existe y rastrear cada fila hasta su origen (linaje).

24. ¿Por qué DimCliente no guarda DNI, RUC ni celular?

No sirven para agrupar ni filtrar ventas y son datos personales; se aplica minimización de datos.

25. ¿Qué es `sp_getapplock` en el SP del calendario?

Un bloqueo con nombre que evita que dos ejecuciones simultáneas inserten fechas a la vez.

26. ¿Cómo se calcula el día de la semana sin DATENAME?

Contando los días desde el 01/01/0001 (lunes) y tomando el residuo entre 7: no depende de DATEFIRST ni del idioma del servidor.

27. ¿Qué es el archivo .ispac?

El proyecto compilado para desplegarlo al catálogo SSISDB y programarlo con el Agente SQL Server.

28. ¿Qué mejorarían?

Reemplazar el Comando OLE DB por staging + UPDATE masivo si crece el volumen; agregar Recuento de filas por dimensión y registrarlo en ETL_Control; agregar miembro desconocido; evaluar SCD 2 para atributos donde la historia importe (rol del empleado, categoría del producto).

5.6 Glosario rápido

Término	Significado
OLTP	Base transaccional (operación diaria). Aquí: Cafeteria.
Data Mart	Base orientada al análisis de un área del negocio. Aquí: DM_Cafeteria.
Esquema galaxia	Varios hechos que comparten dimensiones (constelación de hechos).
Clave natural	ID del sistema origen (`…IDOrigen`).
Clave sustituta	ID propio del DM (`…Key`, IDENTITY).
Desnormalizar	Juntar en una tabla datos que estaban en varias (ej. Categoría dentro de Producto).
SCD Tipo 1	Cambio en una dimensión que sobrescribe el valor anterior.
Upsert	Insertar si no existe, actualizar si existe.
Lookup / Búsqueda	Componente que busca filas del flujo en una tabla de referencia.
División condicional	Componente que envía cada fila a una salida u otra según una expresión.
Comando de OLE DB	Componente que ejecuta una instrucción SQL por cada fila.
Carga rápida (Fast Load)	Inserción masiva del Destino OLE DB.
Contenedor de secuencias	Agrupador de tareas en el flujo de control.
Restricción de precedencia	Flecha entre tareas: Éxito (verde), Error (roja), Finalización (azul).
Idempotente	Que puede ejecutarse varias veces con el mismo resultado.
Columna calculada PERSISTED	Columna cuyo valor calcula SQL Server a partir de otras y lo guarda físicamente.
CTE	Consulta con nombre definida con `WITH`, válida solo en esa instrucción.